1) **Write a program to take a list of 10 farmers fruit they sell ,their cities and the quantity of fruits they have as input. Save the data to notepad before and after sorting based on quantity in tabular manner. Also perform search operation based on farmers name**

```
import java.util.*;    // For Scanner, List, ArrayList, Collections
import java.io.*;      // For File handling

// Class to store farmer details
class Farmer {
String name, fruit, city;
int quantity;

// Constructor to initialize farmer data
public Farmer(String n, String f, String c, int q) {
name = n;
fruit = f;
city = c;
quantity = q;
}
}

public class Lab1 {
public static void main(String[] args) throws IOException {

Scanner sc = new Scanner(System.in);

// List to store Farmer objects
List<Farmer> farmers = new ArrayList<>();

// Taking input (line-by-line like Python style)
for (int i = 0; i < 3; i++) {
System.out.println("\nEnter details for Farmer " + (i + 1));

System.out.print("Enter Name: ");
String name = sc.next();

System.out.print("Enter Fruit Name: ");
String fruit = sc.next();

System.out.print("Enter City: ");
String city = sc.next();

System.out.print("Enter Quantity: ");
int quantity = sc.nextInt();


// Adding farmer object to list
farmers.add(new Farmer(name, fruit, city, quantity)); 33) }


// Save data before sorting
saveToFile(farmers, "Before Sorting");
```

```
// Sorting farmers based on quantity
Collections.sort(farmers, Comparator.comparingInt(f -> f.quantity));


// Save data after sorting
saveToFile(farmers, "After Sorting (by Quantity)");


// Search operation
System.out.print("\nEnter farmer name to search: ");
String search = sc.next();


boolean found = false;


for (Farmer f : farmers) {
if (f.name.equalsIgnoreCase(search)) {
System.out.println("Found: " + f.name + " sells " + f.fruit + " in " + f.city + " (Qty: " + f.quantity + ")"); 47)
found = true;
}
}


if (!found) {
System.out.println("Farmer not found!"); 52) }


sc.close(); // Close scanner
}


// Method to save data into file
static void saveToFile(List<Farmer> farmers, String header) throws IOException {


// true → append mode (data won't overwrite)
PrintWriter pw = new PrintWriter(new FileWriter("farmer_data.txt", true));


pw.println(header);
pw.println("Name\tFruit\tCity\tQty");


// Writing each farmer's data
for (Farmer f : farmers) {
pw.println(f.name + "\t" + f.fruit + "\t" + f.city + "\t" + f.quantity); 64) }

pw.println(); // blank line for separation
```

```
pw.close();   // close file
}
}
```

Input :-

Enter details for Farmer 1
Enter Name:
Kumar
Enter Fruit Name: Grapes
Enter City: Rajkot
Enter Quantity: 52

Enter details for Farmer 2
Enter Name: Rohan
Enter Fruit Name: Orange
Enter City: Goa
Enter Quantity: 34

Enter details for Farmer 3
Enter Name: Ramu
Enter Fruit Name: Strawberry
Enter City: Raipur
Enter Quantity: 17

Enter farmer name to search: Ramu
Output :- Found: Ramu sells Strawberry in Raipur (Qty: 17)

2) Write a program to take quantity and quality of fruits mango,Apple and Papaya as input. Quality mat be good ,bad or average. If entered quantity of fruit is greater than 1000 , store it's s half . Perform search for a given fruit with given quality .

```
import java.util.*;

public class Lab2 {
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Map to store fruit name and quantity
    Map<String, Integer> fruitQty = new HashMap<>();

    // Map to store fruit name and quality
    Map<String, String> fruitQuality = new HashMap<>();

      // Taking input
      for (int i = 0; i < 3; i++) {
      System.out.println("\nEnter details for Fruit " + (i + 1));

      System.out.print("Enter Fruit Name (mango/apple/papaya): ");
      String fName = sc.next();

      System.out.print("Enter Quality (good/bad/average): ");
      String quality = sc.next();
```

```
23)
24)         System.out.print("Enter Quantity: ");
25)         int qty = sc.nextInt();
26)
27)           // If quantity > 1000, store half of it
28)           if (qty > 1000) {
29)           qty = qty / 2;
30)           }
31)
32)         // Store data in maps
33)         fruitQty.put(fName, qty);
34)         fruitQuality.put(fName, quality);
35)         }
36)
37)       // Search operation
38)       System.out.print("\nEnter fruit name to search: ");
39)       String sFruit = sc.next();
40)
41)       System.out.print("Enter quality to search: ");
42)       String sQuality = sc.next();
43)
44)       // Checking if fruit exists and quality matches
45)       if (fruitQuality.containsKey(sFruit)) {
46)
47)        String storedQuality = fruitQuality.get(sFruit);
48)
49)           if (storedQuality.equalsIgnoreCase(sQuality)) {
50)           System.out.println("Found: " + sFruit + " (" + storedQuality + ")");
51)           System.out.println("Quantity: " + fruitQty.get(sFruit));
52)           } else {
53)           System.out.println("Fruit found but quality does not match.");
54)           }
55)
56)         } else {
57)         System.out.println("Fruit not found!");
58)         }
59)
60)       sc.close(); // close scanner
61)       } 62) }
```

Input :-
Enter details for Fruit 1
Enter Fruit Name: mango
Enter Quality: good
Enter Quantity: 1100

Enter details for Fruit 2
Enter Fruit Name: apple
Enter Quality: average
Enter Quantity: 1000

Enter details for Fruit 3
Enter Fruit Name: papaya
Enter Quality: bad

Enter Quantity: 1800

Enter fruit name to search:
mango
Enter quality to search: good
Output :- Found: mango (good) Quantity: 550

3) There are four orchards , namely A,B,CD with fruits in them as Apple ,Litchi,Mango,Apple respectively. The qualities of the fruits in the orchards are good,good, good and bad respectively. The available orchards in a tree view. Search for a given fruit in the available orchards after taking input from the user and print the tree.

```
import java.util.*;

// Class to represent each orchard node class OrchardNode {
String orchardName, fruit, quality;

// Constructor
public OrchardNode(String o, String f, String q) {
orchardName = o; fruit = f; quality = q; } }

public class Lab3 {
public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

// List acts like a tree structure (collection of nodes) List<OrchardNode> tree = new
ArrayList<>();

// Taking input (instead of hardcoding) for (int i = 0; i < 4; i++) {
        System.out.println("\nEnter details for Orchard " + (i + 1));

        System.out.print("Enter Orchard Name: ");
        String orchard = sc.next();

        System.out.print("Enter Fruit Name: ");
        String fruit = sc.next();

        System.out.print("Enter Quality (Good/Bad): ");
        String quality = sc.next();

        // Add node to list
        tree.add(new OrchardNode(orchard, fruit, quality));
}

// Display tree view
System.out.println("\n--- Orchard Tree View ---");
for (OrchardNode node : tree) {
        System.out.println("Orchard " + node.orchardName + " -> "
        + node.fruit + " (" + node.quality + ")"); }

// Search operation
```

```
○ System.out.print("\nEnter fruit to search in orchards: "); ○ String search = sc.next();

○ boolean found = false;

○ for (OrchardNode node : tree) {
        if (node.fruit.equalsIgnoreCase(search)) {
        System.out.println("Found in Orchard " + node.orchardName
        + " with quality: " + node.quality);
        found = true;
        } ○ }

○ // If not found ○ if (!found) {
        System.out.println("Fruit not found in any orchard!");
○ }

○ sc.close(); // close scanner ○ } ○ }
```

Input :-

Enter details for Orchard 1
Enter Orchard Name: A
Enter Fruit Name: Apple
Enter Quality: Good

Enter details for Orchard 2
Enter Orchard Name: B
Enter Fruit Name: Mango
Enter Quality: Good

Enter details for Orchard 3
Enter Orchard Name: C
Enter Fruit Name: Litchi
Enter Quality: Bad

Enter details for Orchard 4
Enter Orchard Name: D
Enter Fruit Name: Apple
Enter Quality: Bad

Enter fruit to search in orchards: Apple
Output:- --- Orchard Tree View --- Orchard A -> Apple
(Good) Orchard B -> Mango (Good) Orchard C -> Litchi
(Bad)
Orchard D -> Apple (Bad)

Found in Orchard A with quality: Good Found in
Orchard D with quality: Bad

4) Take a nap provided as a graph and display all the subtrees of connected points.

```
1) import java.util.*;


2) public class GraphSubtrees {
```

```
// Adjacency list representation using Map
static Map<Integer, List<Integer>> adj = new HashMap<>();

// Function to add edge
static void addEdge(int u, int v) {
adj.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
adj.computeIfAbsent(v, k -> new ArrayList<>()).add(u); 9)          }

// DFS function to print connected nodes
static void dfs(int node, Set<Integer> visited) {
visited.add(node);
System.out.print(node + " ");

if (adj.containsKey(node)) {
for (int neighbor : adj.get(node)) {
if (!visited.contains(neighbor)) {
dfs(neighbor, visited);
}
}
}
}

public static void main(String[] args) {

// Creating graph
addEdge(1, 2);
addEdge(1, 3);
addEdge(4, 5); // separate component

Set<Integer> visited = new HashSet<>();

System.out.println("Subtrees (Connected Components):");

// Traverse all nodes
for (int node : adj.keySet()) {
if (!visited.contains(node)) {
System.out.print("Subtree: ");
dfs(node, visited);
```

```
34) System.out.println();
35) }
36) }
```

Output:-

Subtrees (Connected Components):
Subtree: 1 2 3
Subtree: 4 5

5) Enter entry point and goal point and produce the matching subtrees of the graph. a.

```
Import java.util.*;

b.  Public class EntryGoalGraph {

c.  Static Map<Integer, List<Integer>> adj = new HashMap<>();

d.  Static void addEdge(int u, int v) {
e.  Adj.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
f.  Adj.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
g.  }

h.  // DFS to find path from entry to goal
i.  Static boolean dfs(int curr, int goal, Set<Integer> visited, List<Integer> path) { j.
    Visited.add(curr);
k.  Path.add(curr);

l.  If (curr == goal) return true;

m.  For (int n : adj.getOrDefault(curr, new ArrayList<>())) {
        i.   If (!visited.contains(n)) {
        ii.  If (dfs(n, goal, visited, path)) return true;
        iii. }
n.  }

o.  Path.remove(path.size() – 1); // backtrack
p.  Return false;
q.  }

r.  Public static void main(String[] args) {

s.  addEdge(1, 2);
t.  addEdge(1, 3);
u.  addEdge(2, 4);
v.  addEdge(3, 5);

w.  Scanner sc = new Scanner(System.in);

x.  System.out.print("Enter entry point: ");
y.  Int start = sc.nextInt();

z.  System.out.print("Enter goal point: ");
aa. Int goal = sc.nextInt();
```

```
bb. List<Integer> path = new ArrayList<>();

cc. If (dfs(start, goal, new HashSet<>(), path)) {
        i. System.out.println("Matching Subtree (Path): " + path);
dd. } else {
        i. System.out.println("No path found");
ee. }

ff. Sc.close(); gg. } hh. }
```

Output:- Enter entry point: 1 Enter goal point: 5 Matching Subtree (Path): [1, 3, 5]

6) Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
import java.util.*;

    class Pair {
    int node, weight;
    Pair(int n, int w) {
    node = n;
    weight = w;
    }
    }

public class ShortestPath {

  static Map<Integer, List<Pair>> adj = new HashMap<>();

    // Add weighted edge
    static void addEdge(int u, int v, int w) {
    adj.computeIfAbsent(u, k -> new ArrayList<>()).add(new Pair(v, w));
    adj.computeIfAbsent(v, k -> new ArrayList<>()).add(new Pair(u, w));
    }

  public static void main(String[] args) {

    // Graph with weights
    addEdge(1, 2, 2);
    addEdge(1, 3, 4);
    addEdge(2, 4, 7);
    addEdge(3, 4, 1);
    addEdge(4, 5, 3);

   Scanner sc = new Scanner(System.in);

    System.out.print("Enter start node: ");
    int start = sc.nextInt();

    System.out.print("Enter end node: ");
    int end = sc.nextInt();

    // Dijkstra setup
    Map<Integer, Integer> dist = new HashMap<>();
```

```
40)      for (int node : adj.keySet()) dist.put(node, Integer.MAX_VALUE);
41)
42)     dist.put(start, 0);
43)
44)     PriorityQueue<Pair> pq = new PriorityQueue<>(Comparator.comparingInt(p -> p.weight)); 45)
pq.add(new Pair(start, 0));
46)
47)         while (!pq.isEmpty()) {
48)         Pair curr = pq.poll();
49)
50)        for (Pair neighbor : adj.getOrDefault(curr.node, new ArrayList<>())) { 51)
int newDist = dist.get(curr.node) + neighbor.weight;
52)
53)             if (newDist < dist.get(neighbor.node)) {
54)             dist.put(neighbor.node, newDist);
55)             pq.add(new Pair(neighbor.node, newDist));
56)             }
57)             }
58)             }
59)
60)     System.out.println("Shortest distance from " + start + " to " + end + " is: " + dist.get(end));
61)
62)      sc.close();
63)      }
64)      }
```

Output:- Enter
start node: 1 Enter
end node: 5
Shortest distance
from 1 to 5 is: 8

7) Write a program to demonstrate the use of all keywords made for exception handling.

```
1)   import java.util.*;
2)
3)   class ExceptionDemo {
4)
5)         // Method using "throws" keyword
6)         static void checkAge(int age) throws ArithmeticException {
7)         if (age < 18) {
8)         // "throw" keyword used
9)         throw new ArithmeticException("You are not eligible to vote");
10)        } else {
11)        System.out.println("You are eligible to vote");
12)        }
13)        }
14)
15)   public static void main(String[] args) {
16)
17)     Scanner sc = new Scanner(System.in);
18)
19)        try {
20)        System.out.print("Enter a number: ");
21)        int num = sc.nextInt();
```

```
22)
23)         // Example of exception (division by zero)
24)         int result = 100 / num;
25)         System.out.println("Result: " + result);
26)
27)         System.out.print("Enter your age: ");
28)         int age = sc.nextInt();
29)
30)         // Calling method with "throws"
31)         checkAge(age);
32)
33)       }
34)
35)         // "catch" block
36)         catch (ArithmeticException e) {
37)         System.out.println("Arithmetic Exception: " + e.getMessage());
38)         }
39)
40)         catch (InputMismatchException e) {
41)         System.out.println("Invalid input! Please enter numbers only.");
42)         }
43)
44)         // "finally" block
45)         finally {
46)         System.out.println("This block always executes.");
47)         }
48)
49)        sc.close();
50)        }
51)        }
```

Output:- Enter a number: 0
Arithmetic Exception: / by zero
This block always executes.

Enter a number: 5 Result:
20 Enter your age: 16
Arithmetic Exception: You are not eligible to vote This
block always executes.

8) Write a program to withdraw money from a bank account,if the withdrawn money is greater than the balance, then it throws Money exception .

```
1)    Import java.util.*;
2)
3)        // Custom Exception class
4)        Class MoneyException extends Exception {
5)        MoneyException(String message) {
6)        Super(message);
7)        }
8)        }
9)
10)     Class BankAccount {
11)     Double balance;
```

```
12)
13)     BankAccount(double balance) {
14)     This.balance = balance;
15)     }
16)
17)       // Method to withdraw money
18)       Void withdraw(double amount) throws MoneyException {
19)       If (amount > balance) {
20)       // throw custom exception
21)       Throw new MoneyException("Insufficient balance!");
22)       } else {
23)       Balance = balance – amount;
24)       System.out.println("Withdrawal successful!");
25)       System.out.println("Remaining Balance: " + balance);
26)       }
27)       }
28)       }
29)
30)   Public class Main {
31)   Public static void main(String[] args) {
32)
33)    Scanner sc = new Scanner(System.in);
34)
35)     System.out.print("Enter initial balance: ");
36)     Double balance = sc.nextDouble();
37)
38)    BankAccount obj = new BankAccount(balance);
39)
40)       Try {
41)       System.out.print("Enter amount to withdraw: ");
42)       Double amount = sc.nextDouble();
43)
44)       Obj.withdraw(amount);
45)       }
46)
47)       Catch (MoneyException e) {
48)       System.out.println("Exception: " + e.getMessage());
49)       }
50)
51)       Finally {
52)       System.out.println("Transaction completed.");
53)       }
54)
55)     Sc.close();
56)     }
57)     }
```

Output:- Case 1 Enter initial balance: 5000
Enter amount to withdraw: 2000
Withdrawal successful! Remaining Balance: 3000 Transaction completed.

Case 2
Enter initial balance: 3000 Enter
amount to withdraw: 5000
Exception: Insufficient
balance!
Transaction completed.

9) Write a program which enables to use a platform by a train one at a time.

```
class Platform {

    // synchronized method → only one thread can access at a time
    synchronized void usePlatform(String trainName) {
    System.out.println(trainName + " is entering the platform...");

      try {
      Thread.sleep(2000); // simulate platform usage
      } catch (InterruptedException e) {
      System.out.println(e);
      }

    System.out.println(trainName + " is leaving the platform...");
    }
    }

  // Thread class
  class Train extends Thread {
  Platform platform;
  String trainName;

    Train(Platform p, String name) {
    platform = p;
    trainName = name;
    }

    public void run() {
    platform.usePlatform(trainName);
    }
    }

  // Main class
  public class Main {
  public static void main(String[] args) {

   Platform p = new Platform();

    Train t1 = new Train(p, "Train 1");
    Train t2 = new Train(p, "Train 2");
    Train t3 = new Train(p, "Train 3");

    t1.start();
    t2.start();
    t3.start();
    }
```

```
}
```

Output:-

Train 1 is entering the platform...

Train 1 is leaving the platform...

Train 2 is entering the platform...

Train 2 is leaving the platform...

Train 3 is entering the platform...

Train 3 is leaving the platform...

10) Write a program of restaurant and waiter, which accept order if kitchen is free and chef deliver orders if he is free.

```
class Kitchen {
private boolean isOrderReady = false;


// Waiter places order
synchronized void placeOrder(String order) {
try {
    // Wait if previous order is not yet prepared
    while (isOrderReady) {
    wait();
    }


    System.out.println("Waiter placed order: " + order);
    isOrderReady = true;


    notify(); // notify chef


} catch (InterruptedException e) {
    System.out.println(e);
}
}


// Chef prepares order
synchronized void prepareOrder() {
try {
    // Wait if no order is available
    while (!isOrderReady) {
    wait();
    }

    System.out.println("Chef is preparing the order...");
    Thread.sleep(2000);
```

```
        g.  System.out.println("Chef delivered the order!");
        h.  isOrderReady = false;

        i.  notify(); // notify waiter

12) } catch (InterruptedException e) {
        a.  System.out.println(e);
13) }
14) }
15) }

16) // Waiter Thread
17) class Waiter extends Thread {
18) Kitchen kitchen;

19) Waiter(Kitchen k) {
20) kitchen = k;
21) }

22) public void run() {
23) String orders[] = {"Pizza", "Burger", "Pasta"};

24) for (String order : orders) {
        a.  kitchen.placeOrder(order);
25) }
26) }
27) }

28) // Chef Thread
29) class Chef extends Thread {
30) Kitchen kitchen;

31) Chef(Kitchen k) {
32) kitchen = k;
33) }


34) public void run() {
35) for (int i = 0; i < 3; i++) {
        a.  kitchen.prepareOrder();
36) }
37) }
38) }


39) // Main class
40) public class Main {
41) public static void main(String[] args) {

42) Kitchen kitchen = new Kitchen();

43) Waiter waiter = new Waiter(kitchen);
44) Chef chef = new Chef(kitchen);
```

```
waiter.start();
chef.start();
}
}
```

Output:- Waiter placed order: Pizza
Chef is preparing the order... Chef delivered the order!
Waiter placed order: Burger
Chef is preparing the order... Chef delivered the order!
Waiter placed order: Pasta
Chef is preparing the order... Chef delivered the order!